

ELECTRONICS GUIDE | جزوهٔ آموزشی مهندسی الکترونیک

راهنمای جامع
سه کارکرد هسته‌ای تراشهٔ دولپه‌ای:
HSEM، IPC و PMP

از کارکرد و اصل اجرای نشانه‌گذار سخت‌افزاری، صندوق پیام میان‌هسته‌ای و حفاظت حافظهٔ فیزیکی، تا شیوه‌های استفاده، نمونه‌های کد و نکات برای توسعهٔ دوم

در این جزوه، ترجمهٔ فارسی سند رسمی AN03005 (نسخهٔ ۱٫۰، منتشرهٔ شرکت WCH) را می‌خوانید: نخستین MCU CH32H417 همگانی‌منظورهٔ دولپه‌ای شرکت WCH با معماری دولپه‌ای RISC-V (V5F + V3F) است و سه کارکرد سطح هسته را در خود دارد: نشانه‌گذار سخت‌افزاری (HSEM)، ارتباط میان‌هسته‌ای (IPC) و حفاظت حافظهٔ فیزیکی (PMP). این کارکردها پشتیبانی سخت‌افزاری برای همکاری دو هسته، دسترسی امن به منابع و پایداری سیستم فراهم می‌کنند و بنیان بهره‌گیری از معماری دولپه‌ای‌اند.

تراشهٔ دولپه‌ای CH32H417 | V5F + V3F | نشانه‌گذار سخت‌افزاری HSEM | صندوق پیام IPC

حفاظت حافظهٔ فیزیکی PMP | TOR / NA4 / NAPOT | هم‌گام‌سازی راه‌افت | واژه‌نامهٔ فنی

ترجمهٔ فارسی AN03005 V1.0 · **نسخهٔ ۱٫۰** — آبان ۱۴۰۵

فهرست مطالب — برای پرش به بخش موردنظر، روی ردیف آن کلیک کنید؛ شمارهٔ صفحه‌ها بر اساس صفحات فیزیکی PDF است.

فهرست مطالب

۱ توضیحات و دامنهٔ کاربرد ۳

۲ بخش ۱ | نشانه‌گذار سخت‌افزاری HSEM ۳

1.1 کارکرد و نقش ۳

1.2 اصل کار و استفاده ۴

۳ بخش ۲ | ارتباط میان‌هسته‌ای IPC ۶

2.1 کارکرد و نقش ۶

2.2 اصل کار و استفاده ۷

۴ بخش ۳ | حفاظت حافظهٔ فیزیکی PMP ۹

3.1 کارکرد و نقش ۹

3.2 اصل کار و استفاده ۱۰

۵ تاریخچهٔ نسخه‌ها و بیانیه ۱۴

۶ واژگان کلیدی (واژه‌نامهٔ فنی) ۱۴

فهرست شکل‌ها، جدول‌ها و کدها

کد ۱ هم‌گام‌سازی راه‌افت دو هسته با HSEM ۵

کد ۲ دسترسی متقابل به منابع با HSEM ۶

کد ۳ روند نمونهٔ ارتباط IPC ۸

کد ۴ تنظیم PMP در پردازش وقفهٔ نرم‌افزاری ۱۱

کد ۵ تابع PMP_Set_Range ۱۲

کد ۶ آزمون حفاظت حافظه ۱۳

چکیده — CH32H417 نخستین MCU همگانی‌منظورهٔ دولپه‌ای شرکت WCH است؛ با معماری دولپه‌ای RISC-V (V5F + V3F) و سه کارکرد سطح هسته: نشانه‌گذار سخت‌افزاری (HSEM)، ارتباط میان‌هسته‌ای (IPC) و حفاظت حافظهٔ فیزیکی (PMP). این کارکردها پشتیبانی سخت‌افزاری برای همکاری دو هسته، دسترسی امن به منابع و پایداری سیستم فراهم می‌کنند و بنیان بهره‌گیری از مزایای معماری دولپه‌ای‌اند. این سند کارکرد هسته‌ای، سازوکار اجرا و شیوهٔ استفاده از هر کارکرد را برای مرجع توسعهٔ دوم جمع‌بندی می‌کند.

مقدمه

توضیحات و دامنهٔ کاربرد

CH32H417 نخستین MCU همگانی‌منظورهٔ دولپه‌ای شرکت WCH است؛ با معماری دولپه‌ای RISC-V (V5F + V3F) و سه کارکرد سطح هسته در درونش: نشانه‌گذار سخت‌افزاری (HSEM)، ارتباط میان‌هسته‌ای (IPC) و حفاظت حافظهٔ فیزیکی (PMP). این کارکردها پشتیبانی سخت‌افزاری برای همکاری دو هسته، دسترسی امن به منابع و پایداری سیستم فراهم می‌کنند و بنیان بهره‌گیری از مزایای معماری دولپه‌ای‌اند. این سند کارکرد هسته‌ای، سازوکار اجرا و شیوهٔ پایه‌ای استفاده از هر کارکرد را برای مرجع توسعهٔ دوم جمع‌بندی می‌کند.

جدول ۱ دامنهٔ کاربرد سند

دامنهٔ کاربرد	سری
میکروکنترلرهای همگانی‌منظوره	سری CH32H417

بخش ۱

نشانه‌گذار سخت‌افزاری HSEM

۱.۱ کارکرد و نقش

HSEM (Hardware Semaphore) سازوکار نشانه‌گذاری سخت‌افزارمحور است و عمدتاً مسائل رقابت و هم‌گام‌سازی منابع در سیستم دولپه‌ای را حل می‌کند.

کارکرد هسته‌ای:

- **تک‌دیگرپرهی منابع:** حفاظت از منابع بحرانی مانند جانبی‌های مشترک و حافظهٔ مشترک؛ جلوگیری از به‌هم‌ریختگی داده در نوشتن هم‌زمان دو هسته.
- **هم‌گام‌سازی راه‌افت:** اطمینان از راه‌افت V3F و V5F طبق زمان‌بندی پیش‌بینی‌شده و پرهیز از دسترسی به منابعِ مقداردهی‌نشده.
- **بیداری کم‌مصرف:** آزادکردن نشانه‌گذار می‌تواند وقفه بسازد و هستهٔ دیگر را از حالت STOP/STANDBY بیدار کند.

ویژگی‌ها:

- منطق کاملاً سخت‌افزاری؛ بدون سربار چرخش نرم‌افزاری (spin).
- پشتیبانی از گرفتنِ تک‌دستوری سریع (FastTake).

- پشتیبانی از حالت وقفه؛ آزادسازی نشانه‌گذار به‌طور خودکار وقفهٔ هستهٔ مقابل را می‌سازد.
- کانال‌های نشانه‌گذار مستقلِ متعدد که به منابع مختلف تخصیص‌شدنی‌اند.

◆ ۱.۲ اصل کار و استفاده

◆ ۱.۲.۱ اصل کار

هر نشانه‌گذار دو وضعیت دارد: آزاد (FREE) و اشغال‌شده (TAKEN).

- **گرفتن (Take):** اگر نشانه‌گذار آزاد باشد بلافاصله اشغال و موفقیت بازمی‌گردد؛ اگر اشغال باشد، بسته به حالت، شکست (FastTake غیرمسدودکننده) یا انتظار (حالت مسدودکننده) بازمی‌گردد.
- **آزادسازی (Release):** نشانه‌گذار به وضعیت آزاد برمی‌گردد؛ می‌توان پیکربندی کرد تا وقفه‌ای برای اطلاع‌رسانی به هستهٔ دیگر ساخته شود.

◆ ۱.۲.۲ مهارت‌های استفاده

۱) **هم‌گام‌سازی دولپه‌ای:** V3F نخست اجرا می‌شود، پس از مقداردهی سیستم، V5F را بیدار می‌کند و با WFE وارد انتظار می‌شود؛ پس از راه‌افت، V5F نشانه‌گذار را می‌گیرد و آزاد می‌کند، وقفه ساخته می‌شود و V3F بیدار می‌شود.

```
/******************** کد هستهٔ V3F ********************/
int main(void)
{
SystemInit();

// 0 فعال‌سازی وقفهٔ نشانه‌گذار
HSEM_ITConfig(HSEM_ID0, ENABLE);
// (بدون تابع سرویس وقفه هم بیدار می‌شود) پیکربندی بیداری رویداد
NVIC->SCTLR |= 1 << 4;

// بیدارکردن هستهٔ V5F
NVIC_WakeUp_V5F(Core_V5F_StartAddr);

// ورود به انتظار کم‌مصرف برای آمادهٔ شدن V5F
RCC_HB1PeriphClockCmd(RCC_HB1Periph_PWR, ENABLE);
PWR_EnterSTOPMode(PWR_Regulator_ON, PWR_STOPEntry_WFE);

// به اینجا بیدار می‌شود V5F پس از آزادسازی نشانه‌گذار توسط
HSEM_ClearFlag(HSEM_ID0);

while(1) {
// حلقه اصلی
}
}
/******************** کد هستهٔ V5F ********************/
int main(void)
{
SystemInit();

// 0 گرفتن سریع نشانه‌گذار
HSEM_FastTake(HSEM_ID0);
// را می‌سازد و آن را بیدار می‌کند V3F آزادسازی نشانه‌گذار 0؛ وقفهٔ
HSEM_ReleaseOneSem(HSEM_ID0, 0);

while(1) {
// حلقه اصلی
}
}
```

کد ۱ هم‌گام‌سازی راه‌افت دو هسته با HSEM

۲) دسترسی متقابل به منابع: پیش از دسترسی به جانبی مشترک (مثل SPI، UART) یا حافظهٔ مشترک نشانه‌گذار گرفته می‌شود و پس از پایان کار آزاد می‌شود.

```
// پیش از دسترسی به منبع مشترک
while(HSEM_ReadStatus(HSEM_ID1) != FREE); // انتظار با پرسوجو
HSEM_Take(HSEM_ID1); // گرفتن نشانه‌گذار

// === بخش بحرانی: کار با منبع مشترک ===
// برای مثال: خواندن/نوشتن بافر مشترک، پیکربندی جانبی مشترک

HSEM_Release(HSEM_ID1); // آزادسازی نشانه‌گذار
```

کد ۲ دسترسی متقابل به منابع با HSEM

◆ ۱.۲.۳ نکات

- HSEM_FastTake غیرمسدودکننده است؛ در صورت شکست در گرفتن، بلافاصله بازمی‌گردد و باید مقدار بازگشتی بررسی شود.
- وقفهٔ نشانه‌گذار می‌تواند فقط برای بیداری به‌کار رود و تابع سرویس وقفه نوشته نشود (همراه با بیت بیداری رویداد در SCTLR).
- توصیه می‌شود به منابع مختلف، شناسهٔ نشانه‌گذار متفاوت داده شود و از استفادهٔ تودرتویی که به بن‌بست (deadlock) می‌انجامد پرهیز شود.

بخش ۲

ارتباط میان‌هسته‌ای IPC

◆ ۲.۱ کارکرد و نقش

IPC (Inter-Process Communication) صندوق پیامِ ویژهٔ ارتباط میان‌هسته‌ای است؛ با ۴ کانال ارتباطی مستقل، اطلاع‌رسانی دستور و انتقال دادهٔ کوچک میان دو هسته را فراهم می‌کند.

کارکرد هسته‌ای:

- **اطلاع‌رسانی پیام:** یک هسته به هستهٔ دیگر سیگنال رویداد می‌فرستد و وقفه می‌سازد.
- **انتقال دادهٔ کوچک:** 4×32bit داده؛ مناسب برای فرستادن دستور، وضعیت و اشاره‌گر آدرس.
- **ارتباط دوطرفه:** هر کانال از دریافت و ارسال دوطرفه پشتیبانی می‌کند و وقفه‌ها مستقل‌اند.

تفاوت با حافظهٔ مشترک:

- IPC یک کانال پیامِ سبک با اطلاع‌رسانی وقفه است و برای فرستادن دستورهای کنترلی مناسب است.
- حافظهٔ مشترک (SRAM 512KB مشترک) برای تبادل داده‌های حجمی مناسب است.

- در پروژه‌های واقعی معمولاً ترکیبی استفاده می‌شود: IPC برای اطلاع‌رسانی + حافظهٔ مشترک برای انتقال داده.

۲.۲ اصل کار و استفاده

۲.۲.۱ اصل کار

IPC سازوکار صندوق‌نامه (Mailbox) را به‌کار می‌گیرد:

- شامل ۴ رجیستر دادهٔ ۳۲ بیتی (MSG0~MSG3) است.
- فرستنده پس از نوشتن داده، رجیستر ارسال را فعال می‌کند و وقفهٔ گیرنده ساخته می‌شود.
- گیرنده در وقفه داده را می‌خواند، پس از پردازش فلگ وقفه را پاک می‌کند.
- از دو حالت وقفه پشتیبانی می‌کند: وقفهٔ دریافت و وقفهٔ ارسال.

۲.۲.۲ مهارت‌های استفاده

روند نمونهٔ ارتباط:

```
#ifdef Core_V3F
IPC_DeInit();
IPC_Config(IPC_CH0,IPC_TxCID1,IPC_RxCID0);
IPC_CH0_Lock();
IPC_WriteMSG(IPC_MSG0,0);
IPC_SetFlagStatus(IPC_CH0,IPC_CH_Sta_Bit0);
IPC_ClearFlagStatus(IPC_CH0,IPC_CH_Sta_Bit1);
NVIC_SetPriority(IPC_CH0_IRQn,0<<7);
NVIC_EnableIRQ(IPC_CH0_IRQn);
Delay_Ms(100);
IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit1,ENABLE);

#else
NVIC_SetPriority(IPC_CH0_IRQn,0<<5); //V3 interrupt config
NVIC_EnableIRQ(IPC_CH0_IRQn);

#endif

//تابع سرویس وقفه
#ifdef Core_V3F
if (IPC_GetITStatus(IPC_CH0,IPC_CH_Sta_Bit0) != RESET)
{
uint32_t temp = 0;
temp = IPC_ReadMSG(IPC_MSG0);
printf("temp - %08x\r\n",temp);
temp++;
cnt++;
IPC_WriteMSG(IPC_MSG0,temp);
}
IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit0,DISABLE);

if(cnt < 10)
{
IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit1,ENABLE);
}
else
{
printf("IPC test end\r\n");
}

#else
if(IPC_GetITStatus(IPC_CH0,IPC_CH_Sta_Bit1) != RESET)
{
uint32_t temp = 0;
temp = IPC_ReadMSG(IPC_MSG0);
printf("temp - %08x\r\n",temp);
temp++;
IPC_WriteMSG(IPC_MSG0,temp);
}
IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit1,DISABLE);
```

```
IPC_ITConfig(IPC_CH0,IPC_CH_Sta_Bit0,ENABLE);

#endif
```

کد ۳ روند نمونهٔ ارتباط IPC میان دو هسته

توصیه‌های استفاده:

- **تقسیم کار کانال‌ها:** برای کسب‌وکارهای مختلف کانال مستقل تخصیص دهید (مثلاً کانال ۰ = دستورهای کنترلی، کانال ۱ = اطلاع‌رسانی جریان داده).
- **انتقال دادهٔ بزرگ:** IPC فقط «اطلاع آماده‌بودن داده» و آدرس را می‌فرستد؛ دادهٔ واقعی در SRAM مشترک گذاشته می‌شود.
- **اولویت وقفه:** اولویت وقفهٔ IPC نباید خیلی بالا باشد تا جانبی‌های زمان‌محور مختل نشوند.
- **سازوکار راستی‌آزمایی:** برای دستورهای حیاتی می‌توان کلمهٔ بررسی (check) اضافه کرد تا تداخل به فعال‌سازی اشتباه نینجامد.

بخش ۳

حفاظت حافظهٔ فیزیکی PMP

◆ ۳.۱ کارکرد و نقش

PMP (Physical Memory Protection) سازوکار استاندارد حفاظت حافظهٔ فیزیکی در RISC-V است و می‌تواند برای نواحی حافظه مجوز خواندن، نوشتن و اجرا تعیین کند و از دسترسی غیرمجاز جلوگیری کند. هسته‌های V5F و V3F در CH32H417 هر یک ۴ کانال PMP داخلی دارند.

کارکرد هسته‌ای:

- **جداسازی حافظه:** تقسیم ناحیهٔ کد، ناحیهٔ داده و ناحیهٔ جانبی‌ها با مجوزهای متفاوت.
- **پیشگیری از عملیات اشتباه:** جلوگیری از نوشتهٔ تصادفی برنامه در فلش یا نواحی دادهٔ فقط‌خواندنی.
- **افزایش پایداری:** دسترسی غیرمجاز مانند عبور اشاره‌گر از مرز و سرریز پشته، استثنا می‌سازد نه «پروازِ بی‌صدا».
- **حفاظت امنیتی:** محافظت از رجیسترهای پیکربندی حیاتی در برابر تغییر توسط کدِ غیرمجاز.

توجه: روی بستر CH32H417 فقط هستهٔ V5F از PMP پشتیبانی می‌کند.

۳.۲ اصل کار و استفاده

۳.۲.۱ اصل کار

هر کانال PMP از رجیستر آدرس (pmpaddr) و رجیستر پیکربندی (pmpcfg) تشکیل شده است:

- **رجیستر آدرس:** آدرس آغاز و دامنهٔ ناحیهٔ حفاظت را تعریف می‌کند.
- **رجیستر پیکربندی:** مجوزها (R/W/X)، حالت تطبیق آدرس و بیت قفل را تنظیم می‌کند.

تعریف بیت‌های مجوز:

- R (Read): اجازهٔ خواندن؛ W (Write): اجازهٔ نوشتن؛ X (Execute): اجازهٔ واکشی و اجرا؛ L (Lock): قفل‌کردن پیکربندی تا ریست.

حالت‌های تطبیق آدرس:

- حالت هم‌ترازی طبیعی با دقت ۴ بایت.
- سه حالت پشتیبانی‌شده: TOR (بازهٔ بالایی)، NA4 (ناحیهٔ ۴ بایتی) و NAPOT (ناحیهٔ توان‌دوم هم‌تراز).

۳.۲.۲ مهارت‌های استفاده

استفاده می‌تواند از نمونهٔ رسمی مرجع گرفته شود:

```
#include "hardware.h"

#ifdef Core_V3F
#warning Core V3 is not support PMP
#endif

/* Global define */
#define PMP_MODE_TOR 0x00000008
#define PMP_MODE_NA4 0x00000010
#define PMP_MODE_NAPOT 0x00000018

#define PMP_AUTHOR_R 0x00000001
#define PMP_AUTHOR_W 0x00000002
#define PMP_AUTHOR_X 0x00000004

#define PMP_LOCK 0x00000080
#define PMP_MODE PMP_MODE_NAPOT

/* Global Variable */
volatile uint32_t pmp_grop, pmp_mode, pmp_start_address, pmp_end_address;
volatile uint32_t *pmpaddrbase;
```

```
__attribute__((interrupt("WCH-Interrupt-fast"))) void SW_Handler()
{
if ((pmp_mode >> (pmp_grop * 8) & 0x00000018) == PMP_MODE_TOR)
{
if (pmp_grop == 0)
{
__asm volatile("csrw pmpaddr0 , %0" ::"r"(pmp_end_address));
}
else if (pmp_grop == 1)
{
__asm volatile("csrw pmpaddr0 , %0" ::"r"(pmp_start_address));
__asm volatile("csrw pmpaddr1 , %0" ::"r"(pmp_end_address));
}
else if (pmp_grop == 2)
{
__asm volatile("csrw pmpaddr1 , %0" ::"r"(pmp_start_address));
__asm volatile("csrw pmpaddr2 , %0" ::"r"(pmp_end_address));
}
else if (pmp_grop == 3)
{
__asm volatile("csrw pmpaddr2 , %0" ::"r"(pmp_start_address));
__asm volatile("csrw pmpaddr3 , %0" ::"r"(pmp_end_address));
}
}
else if ((pmp_mode & (PMP_MODE_NA4 << (pmp_grop * 8))) || (pmp_mode & (PMP_MODE_NAPOT <<
(pmp_grop * 8))))
{
if (pmp_grop == 0)
{
__asm volatile("csrw pmpaddr0 , %0" ::"r"(pmp_start_address));
}
else if (pmp_grop == 1)
{
__asm volatile("csrw pmpaddr1 , %0" ::"r"(pmp_start_address));
}
else if (pmp_grop == 2)
{
__asm volatile("csrw pmpaddr2 , %0" ::"r"(pmp_start_address));
}
else if (pmp_grop == 3)
{
__asm volatile("csrw pmpaddr3 , %0" ::"r"(pmp_start_address));
}
}
__asm volatile("csrw pmpcfg0 , %0" ::"r"(pmp_mode));
}
```

کد ۴ تنظیم رجیسترهای PMP در پردازش وقفهٔ نرم‌افزاری

```
void PMP_Set_Range(uint32_t grop, uint32_t md, uint32_t sd, uint32_t ed)
{
#ifdef Core_V5F
#warning "Check the pmp setting carefally"
#endif
pmp_start_address = sd >> 2;
if ((md & PMP_MODE_NAPOT) == PMP_MODE_NAPOT)
{
uint32_t temp = 0xffffffff;
for (int i = 0; i < ed; i++)
{
temp <<= 1;
}
pmp_start_address &= temp;
temp = 0;
for (int i = 0; i < ed - 1; i++)
{
temp <<= 1;
temp |= 0x00000001;
}
pmp_start_address |= temp;
}
else
{
pmp_end_address = ed >> 2;
}

pmp_mode = md << (8 * grop);
pmp_grop = grop;
NVIC_EnableIRQ(Software_IRQn);
NVIC_SetPendingIRQ(Software_IRQn);
}
```

کد ۵ تابع PMP_Set_Range؛ تعیین دامنه و حالت حفاظت

```
volatile uint32_t ProtectSec[0x400] = {1, 2, 3, 4, 5, 6, 7};

uint32_t FinalOprateAddress;
extern void Ecall_M_Mode_Handler(void);

void Hardware(void)
{
#ifdef Core_V3F
printf("Core V3 is not support PMP!!\n");
while(1);
#endif
printf("PMP TEST\r\n");

/* تنظیم دامنه و حالت PMP */
#if PMP_MODE == PMP_MODE_TOR
/*ProtectSec[0x0] - ProtectSec[0x200] قابل نوشتن و اجرا نیست*/
PMP_Set_Range(1, PMP_MODE_TOR | PMP_AUTHOR_R, (uint32_t)(ProtectSec),
(uint32_t)&ProtectSec[0x200]);
#elif PMP_MODE == PMP_MODE_NA4
/*ProtectSec[0] قابل نوشتن و اجرا نیست*/
PMP_Set_Range(0, PMP_MODE_NA4 | PMP_AUTHOR_R, (uint32_t)(ProtectSec), (uint32_t)0);
#elif PMP_MODE == PMP_MODE_NAPOT
/*ProtectSec[0] تا ProtectSec[2^(2+2)] قابل نوشتن و اجرا نیست*/
PMP_Set_Range(0, PMP_MODE_NAPOT | PMP_AUTHOR_R, (uint32_t)(ProtectSec), (uint32_t)2);
#endif

/* با صفر کردن همهٔ عناصر ProtectSec مقداردهی */
FinalOprateAddress = (uint32_t)&ProtectSec[sizeof(ProtectSec) / sizeof(*ProtectSec) - 1];
for (uint32_t z = (uint32_t)&ProtectSec[sizeof(ProtectSec) / sizeof(*ProtectSec) - 1]; z >
(uint32_t)ProtectSec;
z -= 1)
{
FinalOprateAddress = *(uint8_t *)z;
*(uint8_t *)z = 0;
}

printf("Operation succeed!!\r\n");
while (1)
{
Delay_Ms(1000);
}
}
```

کد ۶ آزمون حفاظت حافظه با سه حالت PMP

توجه: مجوز عملیات روی رجیسترهای پیکربندی مرتبط با PMP از نوع MRW است؛ بنابراین باید مطمئن شوید هنگام پیکربندی، به‌درستی وارد حالت ماشین شده‌اید. نمونهٔ رسمی برای سادگیِ نمایش در یک فایل c.، پیکربندی را پس از ورود به main و با ساخت وقفهٔ نرم‌افزاری انجام می‌دهد؛ اما کاربرد متعارف آن است که پیکربندی در فایل راه‌اندازی انجام شود و بعدها به‌ندرت تغییر کند.

پیوست

تاریخچهٔ نسخه‌ها و بیانیه

جدول تاریخچه — تاریخچهٔ نسخه‌های سند اصلی AN03005

تاریخ	نسخه	شرح تغییرات
2026/07/08	V1.0	انتشار نسخهٔ اول

بیانیه

این راهنما صرفاً جهت مرجع ارائه شده است و نویسندهٔ سند اصلی می‌تواند بدون اطلاع قبلی و در هر زمان، تغییراتی در محتوای آن اعمال کند. ترجمهٔ فارسی این سند توسط تیم معمار الکترونیک صنعت تهیه شده است؛ برای اطمینان از به‌روزترین نسخه به https://wch.cn مراجعه کنید.

پیوست

واژگان کلیدی (واژه‌نامهٔ فنی)

اصطلاحات فنی سخت‌افزار مطابق سند اصلی به‌صورت انگلیسی حفظ شده‌اند. جدول زیر معادل‌ها و شرح کوتاه هر واژه را برای مرجع سریع جمع می‌کند.

جدول واژگان — واژگان کلیدی به‌کاررفته در این جزوه

واژه	یعنی چه؟
HSEM	نشانه‌گذار سخت‌افزاری؛ حل مسئلهٔ رقابت و هم‌گام‌سازی منابع میان دو هسته بدون چرخش نرم‌افزاری
FastTake	گرفتن تک‌دستوری سریع نشانه‌گذار؛ غیرمسدودکننده و با بازگشت نتیجه
FREE / TAKEN	دو وضعیت هر نشانه‌گذار: آزاد / اشغال‌شده
IPC	صندوق پیام میان‌هسته‌ای با ۴ کانال مستقل (MSG0~MSG3)؛ برای اطلاع‌رسانی رویداد و انتقال دادهٔ کوچک
Mailbox	سازوکار صندوق‌نامه؛ نوشتن داده، فعال‌سازی رجیستر ارسال و ایجاد وقفه برای گیرنده
PMP	حفاظت حافظهٔ فیزیکی استاندارد RISC-V؛ هر هستهٔ V5F و V3F چهار کانال دارد (فقط V5F پشتیبانی می‌شود)
pmpaddr / pmpcfg	رجیستر آدرس / پیکربندی هر کانال PMP
R / W / X / L	مجوز خواندن / نوشتن / اجرا / قفل پیکربندی تا ریست
TOR / NA4 / NAPOT	حالت‌های تطبیق آدرس: بازهٔ بالای دامنه / ۴ بایت ثابت / توان‌دوم هم‌تراز
SCTLR bit4	بیت «همهٔ وقفه‌های معلق به‌مثابه رویداد»؛ شرط بیداری میان‌هسته‌ای پس از خواب با WFE
WFE	خواب با انتظار رویداد؛ در هم‌گام‌سازی راه‌افت V3F منتظر آزادسازی نشانه‌گذار می‌ماند

واژه	یعنی چه؟
MRW	مجوز پیکربندی رجیسترهای PMP فقط در حالت ماشین؛ پیکربندی باید در نقطهٔ امنِ مد ماشین انجام شود



معمار الکترونیک صنعت

MEMAR ELECTRONIC SANAT (MES)

در مورد مطالب این جزوه و ترجمه، بدیهی است که از نقطه‌نظرات و پیشنهادهای شما عزیزان در این راستا خوشحال می‌شویم. همچنین اشکالات، کم‌وکاستی‌ها و پرسش‌های خود را در این مورد با ما در میان بگذارید تا در نسخه‌های بعدی لحاظ شوند.

با تشکر، حسن معماری

ترجمهٔ فارسی سند AN03005 V1.0 · نسخهٔ ۱٫۰ — تاریخ ساخت: ۱۴۰۵/۰۸/۱۹

راه‌های ارتباط با ما

گیت‌هاب	https://github.com/MEMAR-ELECTRONIC-SANAT
کانال تلگرام	https://t.me/Memar_Electronic
پیج اینستاگرام	https://www.instagram.com/Memar.Electronic
ایمیل	Memari.h@gmail.com